MACHINE LEARNING PROJECT REPORT

Titanic Survival Prediction

Task 3 — Classification Project

A report on data preprocessing, exploratory data analysis, feature engineering, model training, evaluation, and prediction using Python and scikit-learn.

Dataset size	Models trained	Notebook test split
418 rows	3 classifiers	84 rows

Prepared from the supplied Task 3 notebook and README. Reported results are reproduced from those project files.

1. Executive Summary

This project builds machine-learning classifiers to predict whether a Titanic passenger survived. The supplied notebook loads a dataset of 418 records, examines missing values and distributions, creates additional features, encodes categorical variables, trains three classification models, compares their reported accuracy, inspects Random Forest feature importance, and saves the Random Forest model with Joblib.

> The notebook reports 1.00 (100%) accuracy for Logistic Regression, Decision Tree, and Random Forest on an 84-row test split. This is the result recorded in the supplied notebook; it should not be interpreted as proof of real-world performance without independent validation.

Project objectives

• Inspect and prepare passenger data.
• Explore survival, age, fare, and passenger class patterns.
• Create Title, FamilySize, and CabinPresence features.
• Train and compare three classification algorithms.
• Review feature importance and demonstrate a saved-model prediction.

Report sections

Dataset and tools • preprocessing • exploratory analysis • feature engineering • model development • evaluation • inference and saving • limitations and conclusion.

2. Dataset and Tools

The notebook reads a local file named **titanic.csv**. The recorded shape is **418 rows × 12 columns**. The target column is **Survived**, where 0 represents “Not Survived” and 1 represents “Survived”.

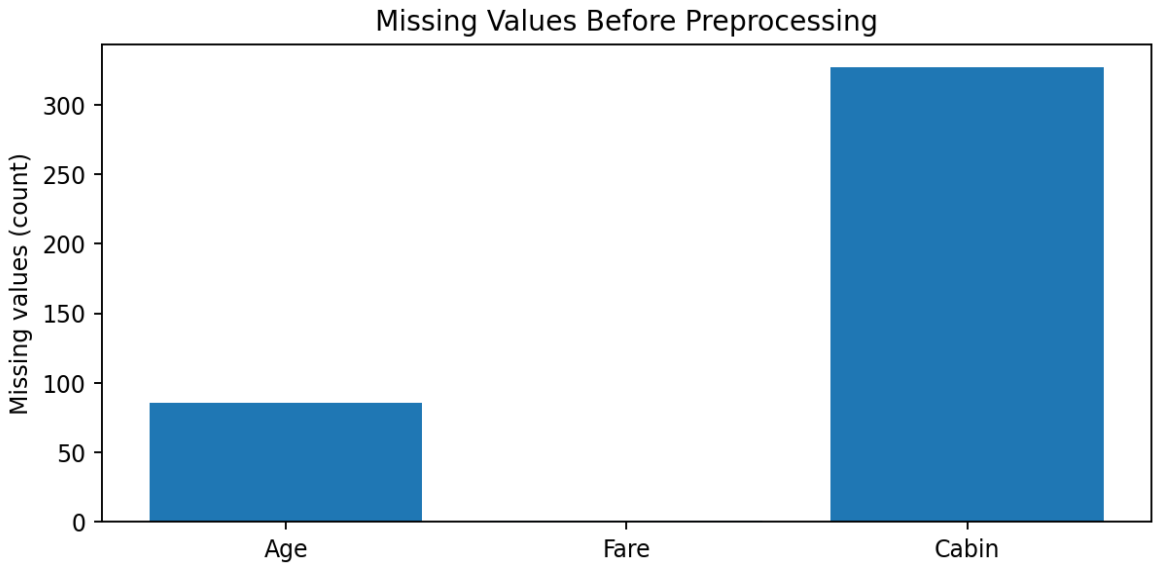
Main columns

Column	Description / role
PassengerId	Passenger identifier
Survived	Target label: 0 = Not Survived, 1 = Survived
Pclass	Passenger class
Name	Passenger name; used to extract Title
Sex	Passenger sex
Age	Passenger age
SibSp / Parch	Sibling-spouse / parent-child counts
Ticket / Cabin	Ticket and cabin information
Fare	Ticket fare
Embarked	Embarkation port

Libraries used

Python, Pandas, NumPy, Matplotlib, Seaborn, scikit-learn, and Joblib. The notebook was prepared for use in an environment such as Google Colab.

Recorded missing values before preprocessing: Age = 86, Fare = 1, Cabin = 327. These values match the notebook output.



3. Data Preprocessing

The notebook applies the following preprocessing steps before training the classifiers:

1. Missing-value handling: missing Age and Fare values are filled using the median of their respective columns.

2. Remove columns: Name, Ticket, and Cabin are dropped after deriving useful information from Name and Cabin.

3. Title grouping: titles are extracted from names. Mr, Miss, Mrs, and Master are retained as common titles; the remaining rare titles are grouped into Other.

4. One-hot encoding: Sex, Embarked, and Title are converted into numeric indicator columns using pandas.get_dummies(drop_first=True).

5. Features and target: Survived is separated as the target (y); all other encoded columns are used as input features (X).

6. Train-test split: an 80/20 split is made with random_state=42 and stratification by target.

Recorded split

Subset	Rows	Feature columns
Training	334	15
Testing	84	15
Total	418	15 input features

Important implementation detail: the notebook includes PassengerId among the model input features. It is an identifier rather than a passenger characteristic, so its use should be reviewed during a future validation pass.

4. Exploratory Data Analysis (EDA)

The notebook includes visualizations for survival counts, age distribution, Age versus Fare by survival, and passenger class versus survival. It also calculates descriptive statistics and checks missing-value counts and percentages.

The dataset summary records an average age of approximately 30.27 years among non-missing Age values and an average Fare of approximately 35.63 among non-missing Fare values. Cabin is missing for 327 of the 418 records (about 78.2%).

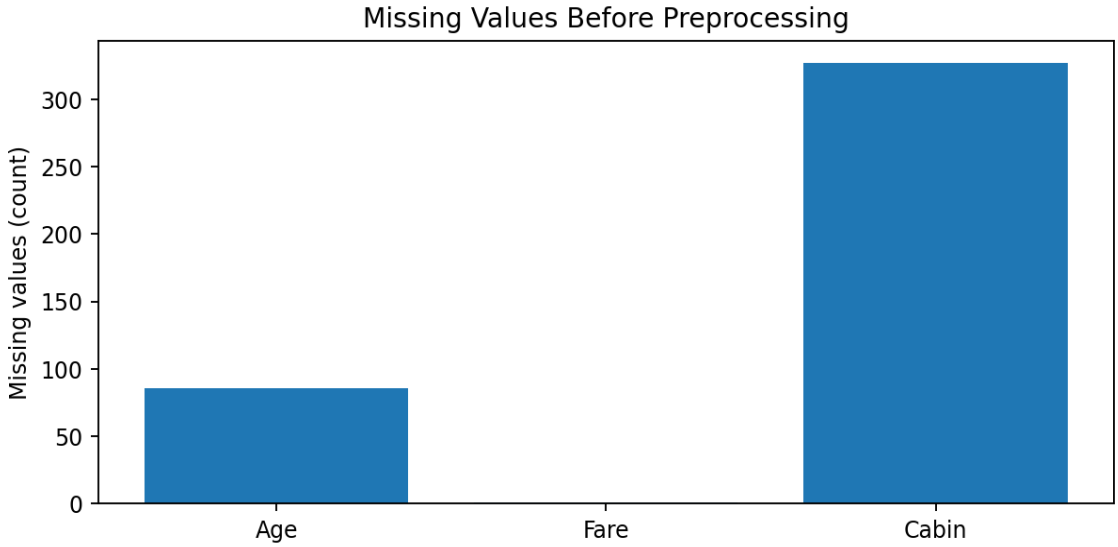
5. Feature Engineering

Title: extracted from the Name field, for example Mr, Mrs, Miss, and Master. Rare titles are combined into Other.

FamilySize: calculated as SibSp + Parch + 1, counting the passenger as part of the family group.

CabinPresence: set to 1 when Cabin is present and 0 when Cabin is missing.

These engineered features are intended to represent social or family-related information that may help classification. The notebook then removes the original Name, Ticket, and Cabin columns.



6. Model Development

Three supervised classification algorithms are trained using the same train-test split.

Logistic Regression

Used as the baseline classifier. The notebook sets max_iter=1000 to allow the solver additional iterations.

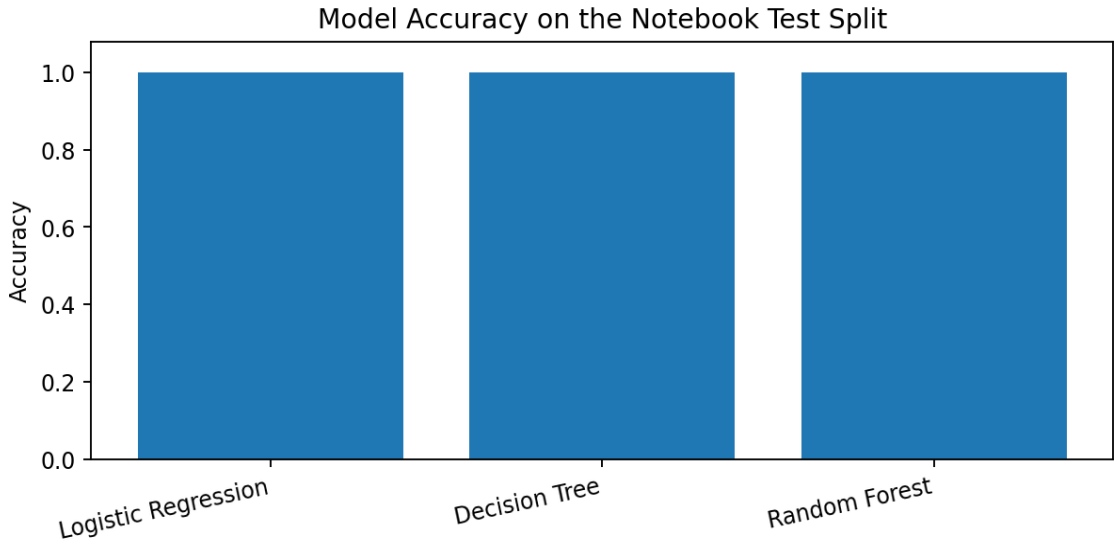
Decision Tree

A tree-based classifier is trained with random_state=42 and max_depth=5.

Random Forest

An ensemble of 100 decision trees is trained with random_state=42 and max_depth=5. The notebook also uses this model to calculate feature importance and to demonstrate prediction with a saved model.

Reported model accuracy



The notebook reports 1.00 accuracy for all three models. For Logistic Regression, the recorded confusion matrix is [[53, 0], [0, 31]]: 53 non-survival cases and 31 survival cases in the test split were classified correctly, according to the notebook output.

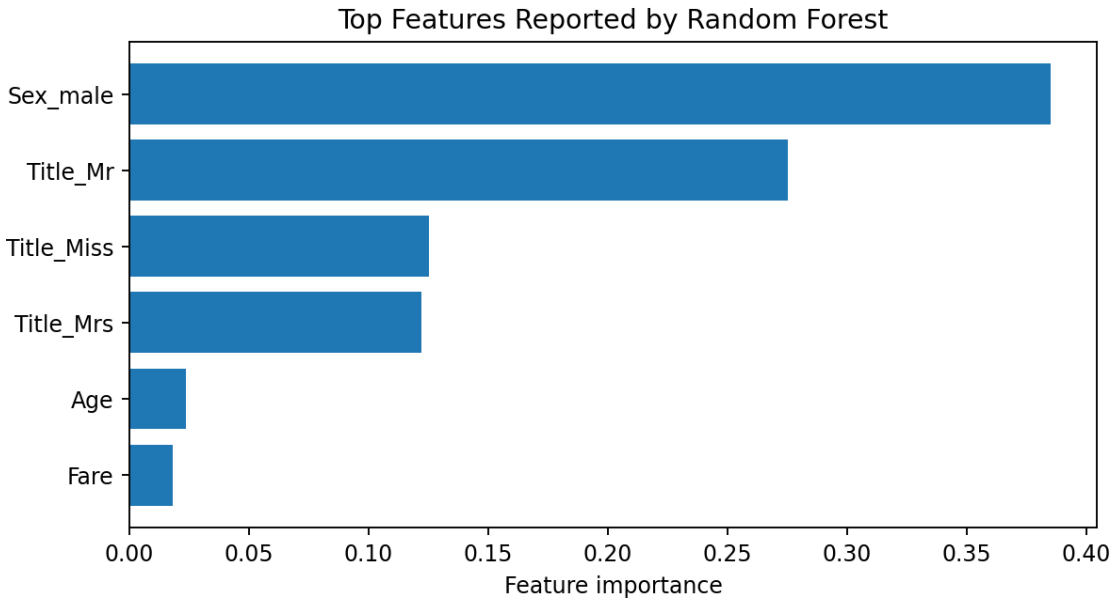
7. Evaluation and Feature Importance

Accuracy is calculated with scikit-learn's accuracy_score. The notebook also prints a classification report containing precision, recall, F1-score, and support, and plots a confusion matrix for Logistic Regression.

Model	Accuracy reported
Logistic Regression	1.00 (100%)
Decision Tree	1.00 (100%)
Random Forest	1.00 (100%)

Random Forest feature importance

The notebook ranks Sex_male as the highest-importance feature (0.3852), followed by Title_Mr (0.2752), Title_Miss (0.1252), and Title_Mrs (0.1220). Age and Fare have smaller importance values in the reported model.



8. Inference and Model Saving

The notebook creates a sample passenger record with Pclass=3, Age=25, SibSp=0, Parch=0, Fare=8.05, FamilySize=1, CabinPresence=0, Sex_male=1, Embarked_S=1, and Title_Mr=1. The remaining encoded indicators are set to 0.

The Random Forest prediction printed by the notebook is **Not Survived**. The model is saved to **titanic_survival_model.pkl** using Joblib, then loaded again and tested on the same sample; the reloaded model returns the same prediction.

Project deliverables listed in the README

• Titanic_Survival_Prediction.ipynb — notebook
• titanic_survival_model.pkl — saved Random Forest model
• requirements.txt — package list
• README.md — project documentation

The notebook creates requirements.txt with pandas, numpy, scikit-learn, matplotlib, seaborn, and joblib.

9. Limitations and Validation Notes

The following points should be reviewed before presenting the reported accuracy as evidence of general predictive performance:

• **Unusually perfect scores:** all three models report 100% accuracy. This may occur in some datasets, but it is unusually strong and should be independently verified.

• **Dataset provenance:** the notebook reports 418 rows and PassengerId values from 892 onward. Confirm the origin and meaning of the supplied titanic.csv file and verify that its Survived labels are appropriate for model training.

• **Identifier feature:** PassengerId is included in X. Compare results after excluding it to check whether it affects the evaluation.

• **Independent validation:** use a verified labelled dataset or cross-validation, and check that no target information is included in the input features.

> These are validation recommendations, not changes made to the supplied notebook. The report preserves the outputs currently recorded in the provided files.

10. Conclusion

The project demonstrates a complete beginner-level classification workflow: data inspection, missing-value handling, feature engineering, categorical encoding, train-test splitting, training multiple models, evaluation, feature-importance analysis, and saving/loading a model for inference.

According to the supplied notebook, Logistic Regression, Decision Tree, and Random Forest each achieved 1.00 accuracy on the selected test split. Random Forest was retained for feature importance and sample inference, where the example passenger was predicted as Not Survived.

The main next step is to verify the dataset labels and repeat evaluation with an independent validation strategy, particularly because every model reports perfect accuracy and PassengerId is used as a feature. This will make the final project more reliable and easier to defend during a viva or internship review.

References / project basis

1. User-supplied file: Titanic_Survival_Prediction_Task_3.ipynb.txt
2. User-supplied file: README.md
3. Tools named in the notebook: pandas, NumPy, scikit-learn, Matplotlib, Seaborn, and Joblib.

End of Report

PROJECT LINKS

GitHub Repository

https://github.com/RushikeshGame/Titanic-Survival-Prediction

Google Colab Notebook

Open Titanic Survival Prediction notebook in Google Colab

Note: The Colab link opens the notebook stored in the GitHub repository. You may need to sign in to Google to run or save your own copy.